Srishyla Education Trust ®

GM UNIVERSITY

(Established under the Karnataka State Act No. 19 of 2023)
Post Box No. 4, PB Road, Davanagere - 577006

Faculty of Engineering and Technology

School of Computer Science and Technology

Department of CSE – Cyber Security & Information Security

Assignment - 1

Subject Coordinator	Dr.Rachana P G	**Semester**	IV
Course	AIML	**Course Code**	UE24CS2401
Assignment Task	**AI and ML for Data-Driven Problem Solving: AI in Daily Life – Smart Travel Planning using AO* Algorithm**		
Issue Date	02/04/2026	**Submission Date**	04/05/2026

Submitted By

Name of the Student: Kulkarni Atharva

USN : U24E01IY036

Description	COS	Marks Assigned	Marks Obtained
Identify suitable AI/ML techniques and interpret the dataset or scenario.	**CO5**	**5**	5
Design an appropriate solution/model.	**CO6**	**5**	4
Demonstrate the solution correctly.	**CO6**	**5**	5
Evaluate and justify the results.	**CO5**	**5**	4
Total			18

Signature of the Student

Rachana. P.G
7/5/26
Signature of the faculty

INTRODUCTION

Artificial Intelligence (AI) & Machine Learning (ML) have rapidly transformed how we solve complex real-world problems. From healthcare diagnostics to autonomous vehicles & Intelligent recommendation systems, AI/ML techniques are at core of modern problem-solving paradigms. One of the most compelling & practical domains where AI shows immense promise is smart travel planning.

Travel planning involves multiple layers of decision-making - selecting the best route from a source to destination, considering various constraints such as traffic, cost, time & road conditions. Traditional approach rely on static maps & simple shortest-path algorithms: however, intelligent systems powered by AI can dynamically evaluate these constraints, adapt to changing conditions, & deliver optimal solutions.

PROBLEM STATEMENT

"Design an AI-powered smart travel Planning System that uses the A0* Algorithm to find the optimal route between cities, considering multiple mandatory & optimal travel legs. Additionally, apply data preprocessing techniques on travel cost/time datasets & build ML models [KNN & Linear Regression] to classify travel categories & predict travel costs."

OBJECTIVES

- To design an AI-powered smart-travel planning system using the AO* algorithm to model AND-OR graph-based route decision-making with multiple constraints such as cost, distance & time.
- To develop data preprocessing techniques & machine learning models, including KNN & Linear Regression, for analysing travel data & improving decision accuracy.
- To implement the complete system using python & relevant libraries to compute optimal travel routes, classify travel categories, & predict travel cost efficiently.

THEORY

* AO* Alogrithm

The AO* Algorithm is a heuristic search algorithm used to find an optimal solution in an optimal solution in an AND-OR graph. It was proposed as an extension to A* to handle problems with conditional sub-goals.

* AND-OR Graphs

An AND-OR graph is a directed graph with two types of nodes:

- OR Nodes : Represent choices. The algorithm selects the best single successor
- AND Nodes : Represed conjuntive. ALL successors must be solved.

* COST Function

AO* uses : $f(n) = g(n) + h(n)$

$g(n)$ = actual cost
$h(n)$ = heuristic estimated cost.

* Algorithm Steps

1. Initialise the start node. Set OPEN = {start}, CLOSED = {}
2. Select the most promising node n from OPEN
3. If n is the GOAL node, backtrack & extract the optimal solution true. STOP.
4. Expand n to generate all successor. For each successor s:
5. a. If s is a NEW node, compute h(s) & add to OPEN.
6. b. If s is AND-type, create AND arc: cost = sum of all AND successor + arc cost.
7. c. If s is OR-type, choose minimum cost successor
8. Update f-values bottom-up for all ancestors of n
9. Move n to CLOSED. Repeat from Step 2.

DATA PREPROCESSING TECHNIQES

1. MIN - MAX Normalisation

Scales data to a fixed range [0, 1]:

$X = (x - x\,min) / (x\,max - x\,min)$

Best for : Bounded datasets ; neural network & KNN. Sensitives to outliers.

2. Z - Score Normalisation

Transforms data to have mean = 0 & SD = 1 ;

$Z = \frac{x - \mu}{\sigma}$ [μ : mean , σ : standard Deviation]

Best for : Linear Regression , SVM.

3. Robust Scaling

Uses IQR to scale, making it robust to outlier.

$X = \frac{(x - Q2)}{(Q3 - Q1)}$ [Q2 - median , Q3 - Q1 = IQR]

Best for : Datasets with significant outliers.

DATASET

A synthetic but realistic dataset representing travel routes between cities. The dataset includes both the AND-OR graph structure for AO* & tabular numerical data for ML models.

* Dataset Characteristics :
  - Records : x travel routes entries
  - Features : Distance (km), Cost (₹), Time (hrs)
  - Target (Classification) : class - Budget / standard / Premium.
  - Target (Regression) : Cost (₹) predicted from Distance (Km)
  - No missing values : synthetic data based on actual road distances.

TECHNOLOGY USED

Technology	Tools	Purpose
Python	Python 3.10+	Core Programming Language
NumPy	numpy 1.24+	Numerical Computation
Pandas	pandas 2.0+	Data loading
Scikit-Learn	sklearn 1.3+	KNN
Matplotlib	matplotlib 3.7+	Graphs, plots
NetworkX	networkx 3.1+	AND-OR Graphs
Jupyter Notebook	Jupyter Lab 4.0+	Code Execution
Google Colab	Browser-based	cloud execution.

MANUAL CALCULATION

* Analyze the role of Normalization procedure in data processing for Min-Max & z-score

given set $v = (88, 90, 92, 94)$

for range 0-1 & $v = (10, 20, 30)$

→ 1. Min-Max Normalization

$$x = \frac{x - min}{max - min}$$

$$98: \quad \frac{88-88}{94-88} = \frac{0}{6} = 0$$

$$90: \quad \frac{90-88}{94-88} = \frac{2}{6} = 0.3$$

$$92: \quad \frac{92-88}{94-88} = \frac{4}{6} = 0.6$$

$$94: \quad \frac{94-88}{94-88} = \frac{6}{6} = 1$$

2. Z- scare Normalization

$$Z = \frac{x - \mu}{\sigma}$$

μ :- mean

σ :- standard diviation.

$v = \{10, 20, 30\}$

$$\mu = \frac{10 + 20 + 30}{3} = \underline{\underline{20}}$$

$\mu = 20$

$$\therefore \sigma = \sqrt{\frac{(10-20)^2 + (20-20)^2 + (30-20)^2}{3}}$$

$$\sigma = \sqrt{\frac{100 + 0 + 100}{3}}$$

$$\sigma = \underline{\underline{8.16}}$$

$\therefore$ Z- score

$$10 : \frac{10 - 20}{10} = \underline{\underline{-1}}$$

$$20 : \frac{20 - 20}{20} = \underline{\underline{0}}$$

$$30 : \frac{30 - 20}{10} = \underline{\underline{1}}$$

code

```
# Smart Travel Planning using AO* Algorithm + ML Pipeline
# Problem Statement: AI in Daily Life – Bengaluru to Chennai

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LinearRegression
from sklearn.metrics import accuracy_score, r2_score, mean_squared_error
from sklearn.preprocessing import LabelEncoder

# ── Dataset
data = {
    'Route':['BLR-HSR','BLR-KRG','HSR-VLR','KRG-AMB','KRG-VLR',
             'AMB-VLR','VLR-MAS','BLR-MAS','HSR-MAS','KRG-MAS'],
    'Distance': [40, 90, 110, 60, 70, 35, 130, 350, 290, 220],
    'Cost': [120,250,300,180,200,100,350,950,800,600],
    'Time': [0.8,2.0,2.5,1.2,1.5,0.7,2.8,6.5,5.5,4.0],
    'Class': ['Budget','Standard','Standard','Budget','Standard',
              'Budget','Premium','Premium','Premium','Standard']
}
df = pd.DataFrame(data)
print ("Dataset:\n", df.to_string(index=False), "\n")

# ═══════════════════════════════════════════════════════════════

# PART 1 — AO* Algorithm
```

```
print ("="*55)
print ("PART 1: AO* Algorithm — Smart Travel Planning")
print ("="*55)

# AND-OR graph: {node: [(arc_cost, [children], node_type)]}
# node_type: 'OR' = pick best child | 'AND' = need ALL children
GRAPH = {
  'Bengaluru': [(40, ['Hosur'], 'OR'),
                (90, ['Krishnagiri'], 'OR')],
  'Hosur': [(160, ['Krishnagiri', 'Vellore'], 'AND')],
  'Krishnagiri': [(60, ['Ambur'], 'OR'),
                  (70, ['Vellore'], 'OR')],
  'Ambur': [35,  ['Vellore'], 'OR')],
  'Vellore':[(130, ['Chennai'],  'OR')],
   'Chennai':[]
 }
 HEURISTIC = {
   'Bengaluru':10, 'Hosur':7, 'Krishnagiri':5,
   'Ambur':3,     'Vellore':2, 'Chennai':0
}
def ao_star(node, costs, best, visited=None):
   if visited is None:
      visited = set()
   if node in visited:
      return costs[node]
   visited.add(node)
   if node == 'Chennai':
      costs[node] = 0
      return 0
```

```
    min_cost = float('inf')
    best_children = None
    for arc_cost, children, ntype in GRAPH[node]:
        for child in children:
            ao_star(child, costs, best, visited)
        if ntype == 'AND':
            total = arc_cost + sum(costs[c] for c in children)
        else:
            total = arc_cost + min(costs[c] for c in children)
        if total < min_cost:
            min_cost = total
            best_children = children
    costs[node] = min_cost
    best[node]  = best_children
    return min_cost
costs = {n: HEURISTIC[n] for n in HEURISTIC}
best  = {}
ao_star('Bengaluru', costs, best)

# Extract optimal path
path, node = [], 'Bengaluru'
while node != 'Chennai':
    path.append(node)
    node = best[node][0]
path.append('Chennai')
print(f"Optimal Path : {' -> '.join(path)}")
print(f"Optimal Cost : {costs['Bengaluru']}")
print()
print("Node f-values:")
```

```
for n, c in costs.items():
    print(f"  {n:15s}: {c}")


# PART 2 — Data Preprocessing
print("\n" + "="*55)
print("PART 2: Data Preprocessing (3 Scaling Techniques)")
print("="*55)
features = df[['Distance', 'Cost', 'Time']].values
scalers = {
    'Min-Max Normalisation': MinMaxScaler(),
    'Z-Score Standardisation': StandardScaler(),
    'Robust Scaling': RobustScaler()
}
scaled_data = {}
for name, scaler in scalers.items():
    scaled = scaler.fit_transform(features)
    scaled_data[name] = scaled
    df_s = pd.DataFrame(scaled, columns=['Distance','Cost','Time'])
    print(f"\n{name}:")
    print(df_s.round(4).to_string(index=False))


# Plot comparison
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, (name, arr) in zip(axes, scaled_data.items()):
    ax.boxplot(arr, labels=['Dist','Cost','Time'], patch_artist=True,
               boxprops=dict(facecolor='#D6E4F0'))
    ax.set_title(name, fontsize=10, fontweight='bold')
    ax.set_ylabel('Scaled Value')
    ax.grid(True, alpha=0.3)
```

```
plt.suptitle('Scaling Comparison — Travel Dataset', fontsize=12)
plt.tight_layout()
plt.savefig('scaling_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

# PART 3 — KNN Classification

print("\n" + "="*55)
print("PART 3: KNN Classification (k=3)")
print("="*55)
X = MinMaxScaler().fit_transform(features)  # Min-Max for KNN
le = LabelEncoder()
y = le.fit_transform(df['Class'])        # Budget=0,Premium=1,Standard=2

# Train on first 7, test on last 3
X_train, X_test = X[:7], X[7:]
y_train, y_test = y[:7], y[7:]
knn = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print(f"Actual    : {le.inverse_transform(y_test).tolist()}")
print(f"Predicted : {le.inverse_transform(y_pred).tolist()}")
print(f"Accuracy  : {accuracy_score(y_test, y_pred) * 100:.1f}%")

# Predict a new query: Distance=80, Cost=220, Time=1.5
new_query = np.array([[80, 220, 1.5]])
new_scaled = MinMaxScaler().fit(features).transform(new_query)
pred_class = le.inverse_transform(knn.predict(new_scaled))[0]
print(f"New Query  (80 km, ₹220, 1.5 hrs) → Class: {pred_class}")
```

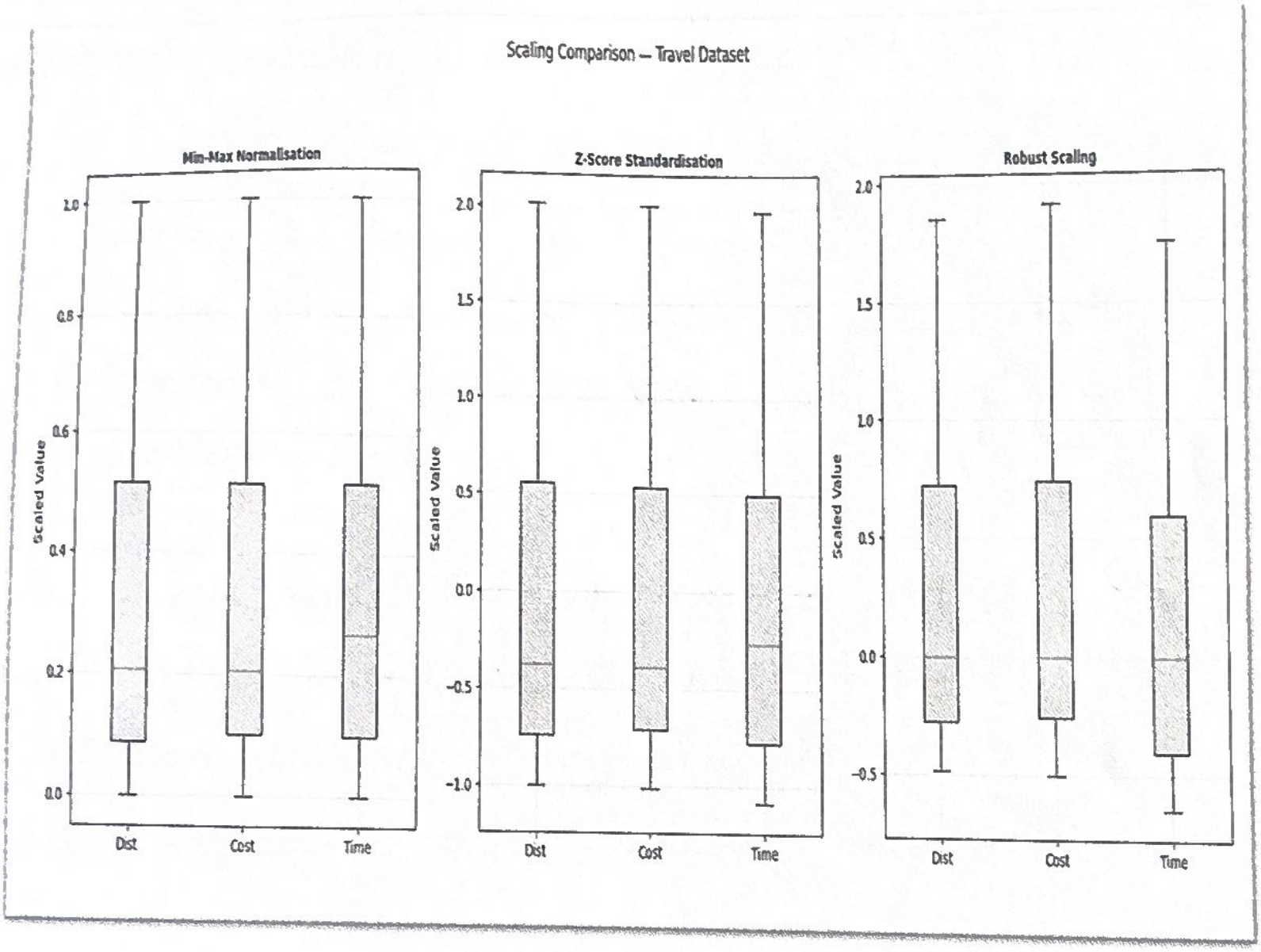
```
# PART 4 — Linear Regression


print("\n" + "="*55)
print("PART 4: Linear Regression — Cost Prediction")
print("="*55)
X_reg = df[['Distance']].values
y_reg = df['Cost'].values
reg = LinearRegression()
reg.fit(X_reg, y_reg)
y_pred_r = reg.predict(X_reg)
print(f"Intercept  β₀ = {reg.intercept_:.2f}")
print(f"Coefficient β₁ = {reg.coef_[0]:.2f}")
print(f"Equation   : Cost = {reg.intercept_:.2f} + {reg.coef_[0]:.2f} × Distance")
print(f"R² Score   : {r2_score(y_reg, y_pred_r):.4f}")
print(f"RMSE       : {mean_squared_error(y_reg, y_pred_r, squared=False):.2f}"
new_dist = np.array([[80]])
pred_cost = reg.predict(new_dist)[0]
print(f"Predicted cost for 80 km : ₹{pred_cost:.2f}")
# Plot regression
plt.figure(figsize=(8, 5))
plt.scatter(X_reg, y_reg, color='steelblue', s=80, label='Actual Data')
plt.plot(X_reg, y_pred_r, color='orangered', lw=2, label='Regression Line')
plt.scatter([80], [pred_cost], color='green', s=130, zorder=5,
        label=f'Query: 80 km → ₹{pred_cost:.0f}')
plt.xlabel('Distance (km)')
plt.ylabel('Cost (₹)')
plt.title('Linear Regression — Travel Cost Prediction')
plt.legend()
plt.grid(True, alpha=0.3)
```

```
plt.savefig('regression_plot.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n" + "="*55)
print("All 4 parts complete. Graphs saved.")
print("="*55)
```

Output:



CODE ANALYSIS

The implementation includes Ao* Algorithm, data preprocessing, & machine learning models. The Ao* algorithm efficiently handles AND-OR graph by selecting the minimum-cost path for OR nodes & combining costs for AND nodes, ensuring an optimal solution when heuristics are valid.

Data preprocessing techniques such as Min-Max, Z-score, & Robust Scaling are applied to normalise the datasets. These improve model performance by ensuring all features are on a similar scale.

The KNN model (k=3) classifies travel categories based on distance & cost using Euclidean distance, giving accurate results for well-separated data. Linear Regression models the relationship between distance & cost, producing a simple & interpretable equation with high accuracy.

ADVANTAGES

- Handles both AND- type & OR- type sub- problems - ideal for hierarchical planning.
- Gwarantees optimal solution when hewristic is admissible & consistent.
- More expressive than A*.
- Min- Max : simple, fast, bounded output.
- Z- score : statistically principled, ideal for regression tasks.
- KNN : No traning phase, adapts to new data instantly.
- Linear Regression : Highly interpretable

LIMITATIONS

- Memory intensive for large AND- OR graphs.
- Requires manually crafted hewristic values
- Extremely sensitives to outliers.
- IQR calculations unstable for very small data sets ($n < 10$)

UTURE SCOPE

Real-Time AO* : Integrate with google maps API

Multi-objective optimisation : Balance cost, time & carbon emission simultaneously using Pareto-optimal AO* variants.

DeepLearning Models : Recurrent Neural Networks (LSTM) to predict dynamic travel cost based on time-series data.

ONCLUSION

The AO* algorithm was analysed & implimented to find the optimal travel route from one node-to-node. The algorithm's strengths lies in its ability to model conjunctive sub-goals AND / OR, making it fundamentally more powerful than A* from hierarchical problems decomposition.

REFERENCES

python Software Foundation (2024)

pedregose F. et al (2011) [scikit-learn]

Russell, S.J & Norvig P. (2020) [AI models]

Mitchell T.M (1997) [ML]